A. Model Setting

We adopt an anisotropic graph neural network (AGNN) architecture as the backbone following Difusco (Sun and Yang 2023). The model learns to operate on edge variables x_t, which are sampled at timesteps t along a probability path from a source distribution $x_0 \sim p_0$ to a target distribution $x_1 \sim p_1$. Given a problem instance s and a specific time t, the model predicts a time-dependent flow field that guides intermediate edge states toward the final solution heatmap representing a valid tour. Compared to conventional GNNs that primarily focus on node-level embeddings, the anisotropic architecture enables more expressive modeling of structural relationships, which is particularly beneficial for combinatorial optimization tasks.

Input Embedding Layer

The input embedding layer encodes node features, edge features, and timestep information to initialize the denoising network.

For TSP, we use the 2D coordinates of nodes as input. Each coordinate is expanded via fixed sinusoidal positional encodings and passed through a linear projection to yield the node embedding $x^0 \in \mathbb{R}^{N\times d}$. For ATSP, node features are extracted from the asymmetric cost matrix by capturing both outgoing and incoming edge information.

For both TSP and ATSP, edge features are constructed by applying sinusoidal scalar encodings to both the pairwise cost matrix and the interpolated edge states x_t, followed by shared linear projections and summation. The denoising timestep t is likewise embedded via fixed sinusoidal functions and mapped through a lightweight multi-layer perceptron (MLP) to yield a global temporal embedding $t^0 \in \mathbb{R}^{d_t}$, which is incorporated into the GNN layers to inform path consistency across the interpolation trajectory.

Graph Convolution Layers

We use a 12-layer AGNN with hidden dimension 256. Each layer updates node and edge features via an anisotropic message passing mechanism that incorporates edge gating and timestep modulation. The update rules are as follows:

$$x_i^{\ell+1} = x_i^\ell + \text{ReLU}\left(\text{BN}\left(W_1^\ell x_i^\ell + \sum_{j\sim i} \eta_{ij}^\ell \odot W_2^\ell x_j^\ell\right)\right),$$

$$e_{ij}^{\ell+1} = e_{ij}^\ell + \text{ReLU}\left(\text{BN}\left(W_3^\ell e_{ij}^\ell + W_4^\ell x_i^\ell + W_5^\ell x_j^\ell\right)\right),$$

where η_{ij}^ℓ is an attention coefficient defined by:

$$\eta_{ij}^\ell = \frac{\sigma(e_{ij}^\ell)}{\sum_{j'\sim i} \sigma(e_{ij'}^\ell) + \epsilon},$$

with σ denoting the sigmoid function and ϵ a small constant. To incorporate the denoising timestep, the edge update is modulated by the global time embedding:

$$e_{ij}^{\ell+1} \leftarrow e_{ij}^{\ell+1} + W_6^\ell \left(\text{ReLU}(t^0)\right).$$

Output Layer

After the final graph convolution layer, we produce an edge heatmap over all directed edges via:

$$\hat{e}_{ij} = \text{Softmax}\left(\text{norm}\left(\text{ReLU}(W_e e_{ij}^L)\right)\right),$$

where L is the total number of layers, W_e is the output projection, and 'norm' denotes layer normalization. The resulting edge-wise distribution is used to decode the predicted tour.

Implementation Details

Our model is implemented using PyTorch and trained on NVIDIA GeForce RTX 4090 GPU. Following (Sun and Yang 2023), we use a learning rate of 0.0002 with a weight decay of 0.0001. The learning rate is scheduled using a cosine decay strategy with an initial linear warm-up over the first 1% of the total training steps. For large-scale problem settings, we implement graph sparsification to reduce computational complexity. We apply graph sparsification by retaining only the k nearest neighbors for each node and augment the graph with convex hull edges to preserve boundary connectivity. For TSP-500, k is set to 50, and for TSP-1000, k is set to 100.

B. Comparison with Diffusion-Based Baselines under Few-Step Inference

In this section, we provide additional results comparing EFLOCO with representative diffusion-based baselines under the same few-step settings. As shown in Table 6, we set the denoising steps of DIFUSCO to two, aligning it with our method for a fair comparison. For T2T, which involves both denoising and rewriting operations, we configure it with two denoising steps followed by one rewriting step. Under this setting, our method achieves nearly all the best performance within just two steps, demonstrating both its effectiveness and efficiency. While T2T achieves competitive results on certain instances, its dependence on the time-consuming rewriting stage limits its overall efficiency.

C. Inference Time Analysis for ATSP

We provide a detailed analysis of the time composition of our method for ATSP to better understand how computation is distributed across different stages of the inference pipeline. Specifically, we decompose the total runtime into hierarchical components: the network inference, the network inference and decoding, and the full pipeline time. This layered breakdown helps reveal the relative time consumption of each stage and offers insights into the efficiency and scalability of EFLOCO under various step counts and problem sizes.

As shown in Figure 4, we observe that decoding time constitutes a significant portion of the total runtime, particularly when the number of inference steps is small. For instance, with only 2 steps, the forward network takes just 0.32 minutes, whereas the decoding stage increases the total time to 3.8 minutes—implying that decoding dominates the overall cost in this regime. As the number of steps increases (e.g.,

Method	TSP-50		TSP-100		TSP-500		TSP-1000	
	Step	Gap(%) ↓	Step	Gap(%) ↓	Step	Gap(%) ↓	Step	Gap(%) ↓
DIFUSCO	2	0.20	2	0.57	2	1.89	2	2.10
T2TCO	2 + 1	0.08 (+11.3m)	2 + 1	**0.32** (+11.8m)	2 + 1	1.58 (+2.01m)	2 + 1	2.04 (+4.02m)
EFLOCO	2	**0.07**	2	0.38	2	**1.48**	2	**1.73**

Table 6: Performance comparison on TSP. Lower gap indicate better performance. **Bold** indicates the best solution; underlined indicates the second-best solution. The time in parentheses denotes the additional runtime of T2T compared to 2-step DIFUSCO and EFLOCO. Since DIFUSCO and EFLOCO have similar runtimes under the same step count, we omit the individual timings for clarity.

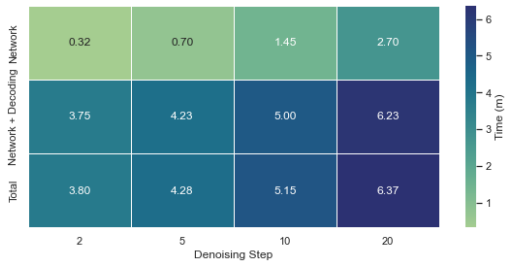


Figure 4: Runtime decomposition of EFLOCO under different inference steps. The total runtime is partitioned into three hierarchical components: (1) **Network Inference**, representing the time spent on forward computation; (2) **Network Inference + Decoding**, which includes network inference and the subsequent decoding process; and (3) **Total Runtime**, encompassing all stages including cost evaluation.

Problem	Train Set Size	Test Set Size
TSP-20	1,502,000	1280
TSP-50	1,502,000	1280
TSP-100	1,502,000	1280
TSP-500	128,000	128
TSP-1000	64,000	128
ATSP-20	128,000	1280
ATSP-50	128,000	1280
ATSP-100	128,000	1280

Table 7: Training and test set sizes for EFLOCO on different problem types.

to 10 or 20), the relative overhead of decoding diminishes, and the forward computation becomes the primary contributor. This layered breakdown helps reveal the relative time consumption of each stage and demonstrates that EFLOCO performs inference efficiently; further improvements in the decoding stage could lead to even faster overall runtimes.

D. Training and Test Datasets

In this section, we present the training and testing datasets used for each problem type. The TSP configurations follow standard setups commonly adopted in recent diffusion-based methods (Sun and Yang 2023; Li et al. 2023), while the size of ATSP train dataset is constructed following the setting introduced in (Hong et al. 2025).

E. Decoding Strategies

Greedy Strategy

The decoding procedure constructs a complete solution from the model's output heatmap using a greedy traversal strategy. For TSP, this involves sequentially selecting the highest-probability node or edge at each step and appending it to the partial tour until all nodes are visited. For ATSP, the process starts from an initial node node and iteratively selects the outgoing directed edge with the highest transition probability to an unvisited node, thereby constructing an initial tour. Since this greedy process does not inherently ensure a valid Hamiltonian cycle, a manual closure step is applied by appending a return edge from the final node back to the starting point. To further enhance solution quality, the decoded tour is refined using local search heuristics, such as 2-opt for symmetric problems or node relocation for asymmetric settings.

Sampling Strategy

Following (Kool, van Hoof, and Welling 2019), we adopt a sampling scheme where multiple candidate solutions are generated in parallel by drawing samples of the initial state x_0 from the base distribution. Each candidate is decoded using the same greedy algorithm described above, and the best-scoring solution among them is selected as the final output. Increasing the number of samples typically enhances candidate diversity and improves the likelihood of discovering better solutions.

F. Visualization of Interpolation Process

We visualize the forward interpolation process in EFLOCO, which progressively perturbs the optimal solution by sampling intermediate states between the structured solution and the base distribution. We consider TSP instances of size 20, 50, and 100, and generate the interpolated states according to Eq. (3). The interpolated states are visualized both as graphs and adjacency matrices. As shown in Figure 5, Figure 6 and Figure 7, as the interpolation time t increases from 0 to 1, the adjacency matrices gradually evolve from unstructured, near-uniform patterns to well-formed solutions. This reflects the generative process of EFLOCO, where samples drawn from the base distribution are transformed into structured TSP solutions through the learned flow.

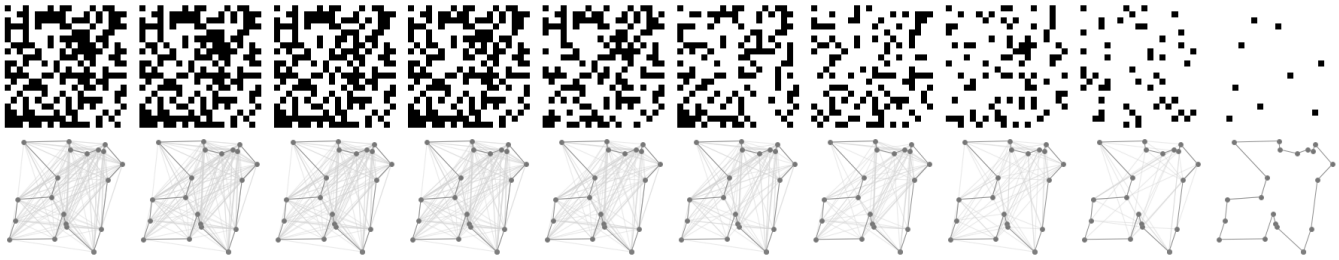


Figure 5: Visualization of the interpolation process in EFLOCO for TSP-20. Each row illustrates the evolution of adjacency matrices (upper) and corresponding graph structures (lower) as the interpolation time t increases from 0 to 1.

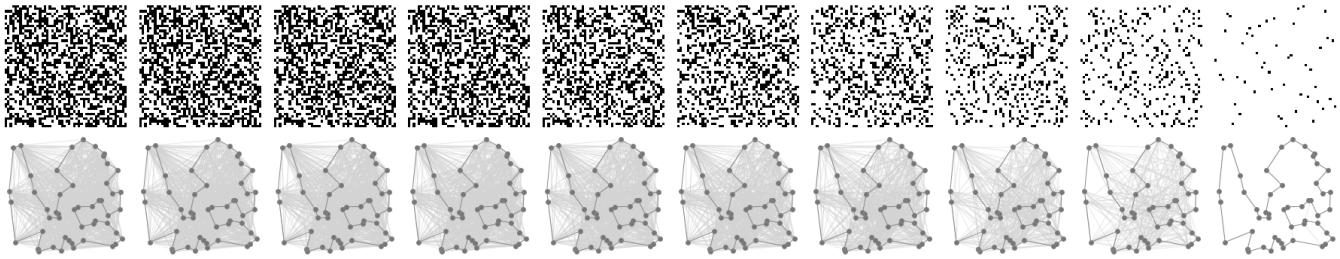


Figure 6: Visualization of the interpolation process in EFLOCO for TSP-50.

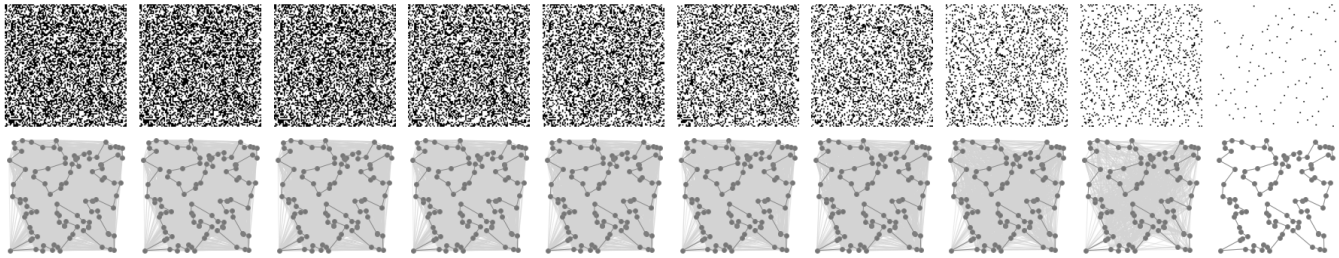
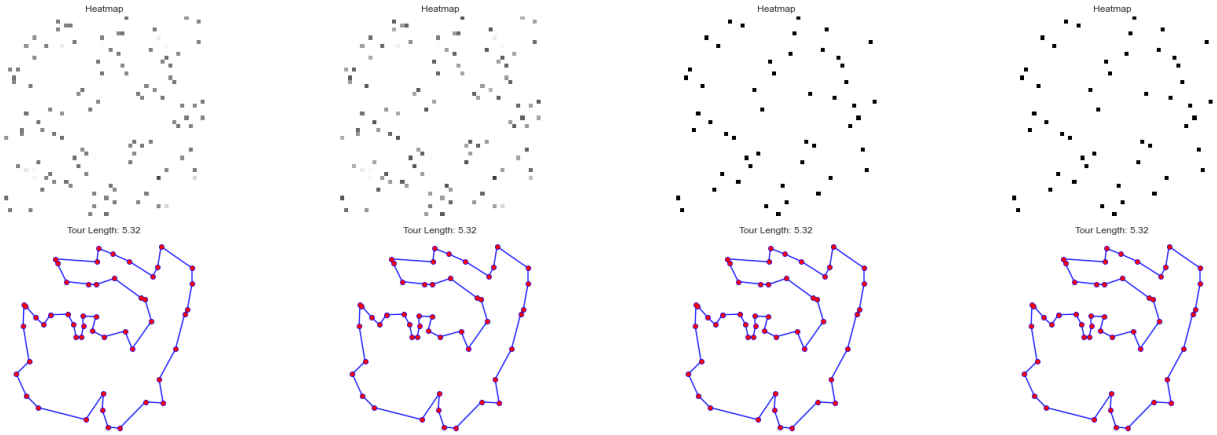


Figure 7: Visualization of the interpolation process in EFLOCO for TSP-100.

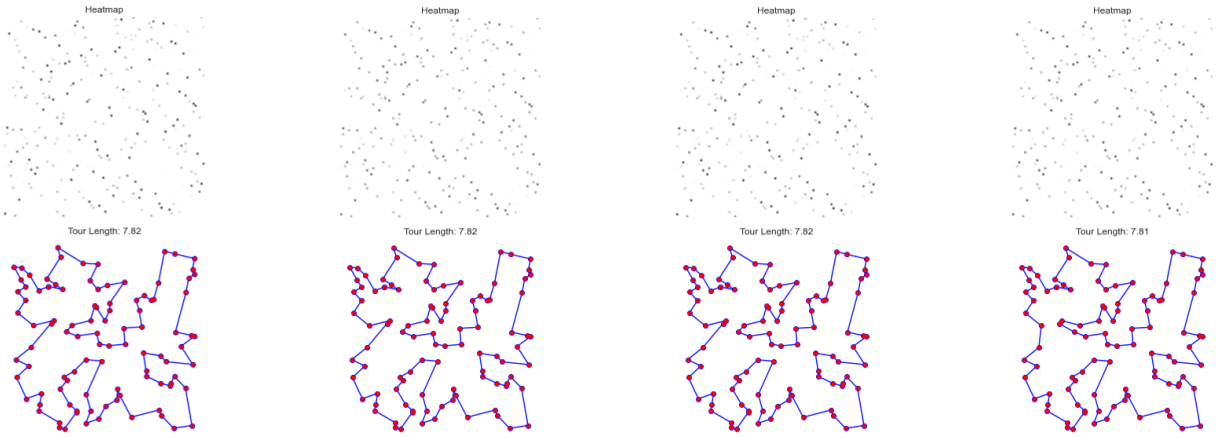
G. Visualization of Generated Paths

In this section, we visualize the final heatmaps and the corresponding decoded tours generated by our method under different inference steps. To ensure clarity and facilitate comparison, we select representative instances from the TSP-50 and TSP-100 datasets. As shown in Figure 8 and Figure 9, each heatmap corresponds to a complete solution decoded from the model's output, with the resulting tour length annotated below the heatmap image. These visualizations show the heatmaps and solutions generated under different inference steps.



(a) Step 2 (b) Step 5 (c) Step 10 (d) Step 20

Figure 8: Visualization of the generated heatmaps and decoded paths on representative instances of TSP-50.



(a) Step 2 (b) Step 5 (c) Step 10 (d) Step 20

Figure 9: Visualization of the generated heatmaps and decoded paths on representative instances of TSP-100.